WGXCoders DC

An empowering community where women and non-binary individuals develop technical skills, share experiences, and build friendships.

We welcome all supporters while maintaining focus on the experiences of underrepresented groups in tech.

Please review our Code of Conduct.

Hosting Sponsor

Thank you to Prefect for hosting this workshop!

Before we start

Please create an account on TestPyPI and **enable 2FA**.

We will use it later to publish your package.

WGCX-DC Python Packaging Workshop

Based on the PyOpenSci Python Packaging 101 Tutorial

Workshop repo: github.com/wgxcodersdc/python-packaging

What is a Python package?

A **directory with a specific structure** that contains Python code organized into modules (`.py` files).

You already use packages every day:

```
import numpy as np
import pandas as pd
```

Today, you'll learn how to **create your own**.

Why create a package?

- **Reuse** your code across multiple projects
- **Share** your work with others via PyPI or GitHub
- **Collaborate** with the community through open source
- **Organize** large codebases into manageable pieces

> PyOpenSci intro

What makes up a Python package?

Component	Purpose
Code	Functions and classes
Tests	Verify it works correctly
Documentation	Help users and contributors
License	Legal terms for reuse
Infrastructure	CI/CD, automation

What we'll cover

1. Create a Python package using a template
2. Understand the package structure (`src/` layout, `__init__.py`, `pyproject.toml`)
3. Install and test it locally
4. Add dependencies and write tests
5. Push to GitHub and install from a repository
6. Build and publish to TestPyPI

What is a virtual environment?

An **isolated Python installation** where you can install packages without affecting your system Python or other projects.

```
uv venv
source .venv/bin/activate
```

Each project should have its own virtual environment.

The tools we'll use

- **Copier** — scaffolds a package from a template
- **Hatch** — manages environments, builds, and publishing
- **uv** — fast Python package installer
- **pytest** — testing framework

All pre-installed in the Codespace. See the Setup instructions to fork the repo and configure the devcontainer.

Step 1: Scaffold with copier

```
copier copy gh:pyopensci/pyos-package-template .
```

The template creates a complete package structure with tests, docs, and configuration — all following best practices.

Package directory structure

After running the copier template, your package will have this among other files that we won't cover due to time. Note: `wgxcpack` is the package name I chose — you should see the name you picked during setup.

```
wgxc-package/
├── src/
│   └── wgxcpack/
│       ├── __init__.py
│       └── example.py
├── tests/
│   └── test_example.py
├── pyproject.toml
└── README.md
```

The `src/` layout

- Your code lives under `src/<package_name>/`
- This is the **src layout** — it prevents accidentally importing the local directory instead of the installed package
- Explore `src/wgxcpack/example.py` to see the example functions

> PyOpenSci package structure best practices

What is `__init__.py`?

Marks a directory as a **Python package**.

- Runs when you `import wgxcpack`
- Can be empty — that's fine!
- Can define what gets exported when someone imports your package

Without this file, Python won't recognize the directory as importable.

What is `pyproject.toml`?

The **single configuration file** for your entire package.

It replaces the old `setup.py`, `setup.cfg`, and `requirements.txt` approach.

Defined by PEP 621.

`pyproject.toml` — project metadata

```toml
[project]
name = "wgxcpack"
version = "0.1.0"
requires-python = ">=3.11"
```

- **name** — how users `pip install` your package
- **version** — follows semantic versioning
- **requires-python** — which Python versions are supported

Step 2: Install your package

```
uv venv
source .venv/bin/activate
uv pip install -e .
```

The `-e` flag means **editable mode**: changes to your source code take effect immediately without reinstalling.

Step 3: Test your package

```
>>> import wgxcpack
>>> from wgxcpack.example import add_numbers
>>> add_numbers(1, 2)
3
```

If this works, your package is installed correctly!

What is pytest?

A testing framework that makes it easy to write and run tests.

```python
def test_add_numbers():
    """Test that add_numbers works as expected."""
    out = add_numbers(1, 2)
    expected_out = 3
    assert out == expected_out, f"Expected {expected_out} but got {out}"
```

- Tests live in the `tests/` directory — **not** inside `src/`. You usually don't want to package your tests with your library.
- Run with `pytest`
- Measure coverage with `pytest --cov=wgxcpack`

See the testing steps in the README for the full walkthrough.

Step 4: Add a dependency

Edit `pyproject.toml`:

```
dependencies = ["numpy>=2.0"]
```

Then reinstall:

```
uv pip install -e .
```

Now you can `import numpy` in your package code.

Step 5: Push to GitHub and make it installable

```
git add .
git commit -m "add first python package"
git push origin first-python-package
```

In general, once your code is on GitHub, anyone can install it directly:

```
pip install git+https://github.com/<user>/<repo>.git@branch
```

Let's follow the instructions for our case full instructions in the README.

What is PyPI?

The **Python Package Index** — the official repository for Python packages.

- `pip install numpy` fetches from PyPI
- **TestPyPI** is a separate instance for testing
- You need an account and API token to publish

> PyOpenSci publishing guide

`pyproject.toml` — build system

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Tells tools **how** to build your package.

hatchling is the build backend used by the pyOpenSci template. Other options include setuptools, flit, and meson-python.

`pyproject.toml` — dependencies

```toml
[project]
dependencies = ["numpy>=2.0"]
```

```toml
[project.optional-dependencies]
tests = ["pytest", "pytest-cov"]
```

- **dependencies** — required at runtime
- **optional-dependencies** — installed with `pip install ".[tests]"`

`pyproject.toml` — hatch environments

```toml
[tool.hatch.envs.build]
dependencies = [
    "hatchling",
    "pip-audit",
    "twine",
]

[tool.hatch.envs.build.scripts]
check = [
    "pip check",
    "hatchling build {args:--clean}",
    "twine check dist/*",
]
```

Custom environments let you run tasks like building, linting, or publishing.

What are sdist and wheel?

Building a package turns your source code into two **distribution files**:

- **wheel** (`.whl`) — a pre-built, ready-to-install format. This is what `pip install` grabs first because it's fast — no build step needed.
- **sdist** (`.tar.gz`) — source distribution containing your unbuilt project files. Important if you want to publish to conda-forge later.

You should always publish **both** to PyPI.

> PyOpenSci — build your sdist and wheel

Step 6: Publish to TestPyPI

The high-level steps:

1. Disable keyring (needed in Codespaces)
2. Build your sdist and wheel
3. Publish to TestPyPI
4. Verify by installing in a fresh environment

Follow the detailed instructions in the README.

Verify your published package

```
uv venv new-env
source new-env/bin/activate
uv pip install \
  --index-url https://test.pypi.org/simple/ \
  --extra-index-url https://pypi.org/simple/ \
  wgxcpack
```

The extra index is needed so pip can find dependencies (like NumPy) on the real PyPI.

What we built today

1. Scaffolded a package with copier
2. Learned about `pyproject.toml`, `src/` layout, and `__init__.py`
3. Installed and tested locally
4. Added dependencies and wrote tests
5. Published to TestPyPI

Next steps

- Add documentation with Sphinx or MkDocs
- Set up CI/CD with GitHub Actions
- Use trusted publishing for automated PyPI uploads
- Add type hints and linting

> PyOpenSci Python Package Guide

Resources

- PyOpenSci Packaging 101
- PyOpenSci Package Structure Guide
- PyOpenSci Publishing Guide
- PyOpenSci Trusted Publishing
- Python Developer's Guide — Supported Versions
- PyOpenSci Python Package Guide

WGXC DC

Thank you!

wgxcodersdc.org